A.1 MATLAB CODE OF WELGE GRAPHIC METHOD

Problem description

Analytical solution of Buckley–Leverett equation using Welge method Brooks–Corey type relative permeability curves are used

```
%close all; clear all; clc;
```

1 Parameters initialization

1.1 rock properties

```
L = 100.0; %    domain length [m]
A = 1.0; % area of cross-section [m^2]
phi = 0.25; % porosity
k = 9.869e-12; % absolute permeability [m^2]
theta = pi * 0.0; % angle: x-direction vs horizontal[rad]
```

1.2 fluid properties

```
muo = 5.0e-3; % oil phase viscosity [Pa*s]
muw = 1.0e-3; % water phase viscosity [Pa*s]
rho_o = 0.8e3; % oil density [kg/m^3]
rho_w = 1.0e3; % water density [kg/m^3]
dlt_rho = rho_w - rho_o; % density difference [kg/m^3]
```

1.3 relative permeability parameters

```
Sor = 0.10; % residual oil saturation
Swc = 0.10; % connate water saturation
no = 1.00; % exponent of oil phase
nw = 2.00; % exponent of water phase
kro_max = 0.80; % maximum permeability of oil phase
krw_max = 0.80; % maximum permeability of water phase
```

1.4 other initial parameters

```
dlt_Sw = 1e-3; % constant saturation step
Sw = [(Swc + eps):dlt_Sw:(1 - Sor)]'; % water saturation vector
qt = 1.0e-4; % constant water injection rate [m^3/s]
g = 9.8067; % gravity acceleration constant [m/s^2]
time0 = 86400 * 0.1; % initial calculation time [s]
timef = 86400 * 1.0; % final calculation time [s]
nts = 4; % time steps for calculating
nsw = size(Sw, 1); % number of water saturation vector
format = '%4.2e'; % precision format for plotting legend
```

2 Relative permeabilities, fractional flow function and its derivative

2.1 initialization of function handles

```
kr_w = @(S) krw_max .* ((S - Swc) / (1 - Swc - Sor)) .^ nw; % relative permeability of water
kr_o = @(S) kro_max .* ((1 - S - Sor) / (1 - Swc - Sor)) .^ no; % relative permeability of oil
mob = @(S) (kr_o(S) ./ muo) .* (muw ./ kr_w(S)); % mobility function
f_w = @(S) 1 ./ (1 + mob(S)) - A * k .* kr_o(S) ./... % fractional flow function fw
(qt * muo) * dlt_rho * g * sin(theta) ./ (1 + mob(S));
df_w = @(S) ((nw .* mob(S) ./ (S - Swc)) + (no .* mob(S))... % derivate of fw
./ (1 - S - Sor)) ./ (1 + mob(S)) .^ 2 + A * k .* kr_o(S) * no * dlt_rho * g * sin(theta) ./...
((1 - S - Sor) * qt * muo .* (1 + mob(S))) + A * k .* kr_o(S) * dlt_rho * g * sin(theta) ./...
(qt * muo .* (1 + mob(S)) .^ 2) .* (((nw .* mob(S) ./ (S - Swc)) + (no .* mob(S)) ./...
(1 - S - Sor)) ./ (1 + mob(S)) .^ 2);
```

2.2 relative permeability and fractional flow vectors

```
krw = kr_w(Sw); % relative permeability vector of water
kro = kr_o(Sw); % relative permeability vector of oil
fw = f_w (Sw); % fractional flow vector
dfw = df_w(Sw); % fractional flow derivate vector
```

3 Calculate advance frontal water saturation

3.1 Evaluate advance frontal water saturation Swf

```
[index, Swf, dfwf, bf] = calSatFront(Sw, fw, dfw);
```

3.2 Calculate the time when Swf reaches at production well

```
krw_swf = kr_w(Swf);
```

```
kro_swf = kr_o(Swf);
fw_swf = f_w (Swf);
dfw_swf = df_w(Swf);
t_pw = A * phi * L / (qt * dfw_swf);
```

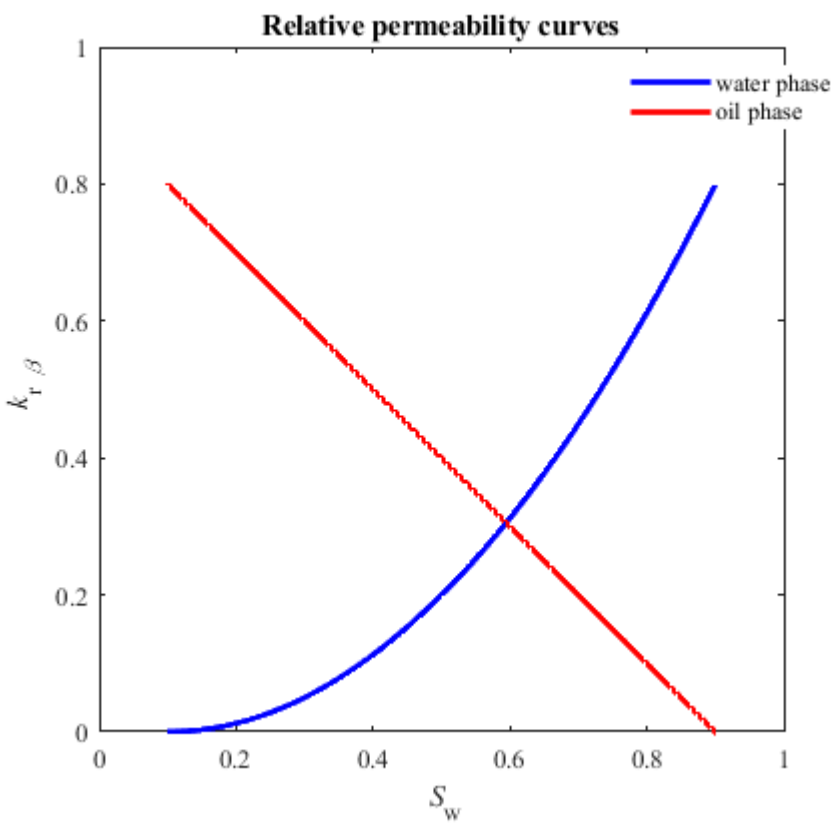
4 Calculate water saturation profile

```
t = linspace(time0, timef, nts);
dfwt = dfw(end:-1:index); % inverted sequence of dfw
Swt = Sw(end:-1:index); % inverted sequence of Sw

for ti = 1:nts

    for i = 1:(nsw - index + 1)
        Xsw(i, ti) = qt * t(ti) / (A * phi) * dfwt(i);
    end

end
```

5 Plot results

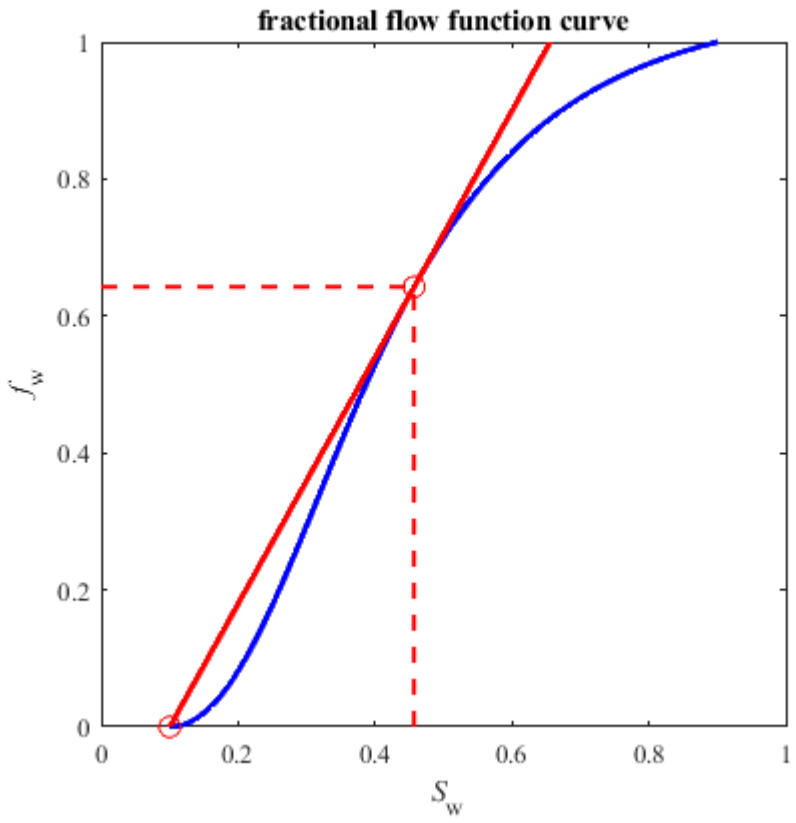
5.1 Plot the relative permeability curves

```
h_fig1 = figure(1);
set(h_fig1, 'color', 'w', 'NumberTitle', 'off', 'Name', 'Relative Permeability Curves:kr');
plot(Sw, krw, '-b', Sw, kro, '-r', 'LineWidth', 2.0);
axis([0.0 1.0 0.0 1.0]);
axis square;
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('Relative permeability curves')
xlabel('{\it S}_w');
ylabel('{\it k}_{r {\it \beta}}');
set(gca, 'YTick', 0:0.2:1);
h_legend1 = legend('water phase', 'oil phase');
set(h_legend1, 'Box', 'on', 'Location', 'best');
```



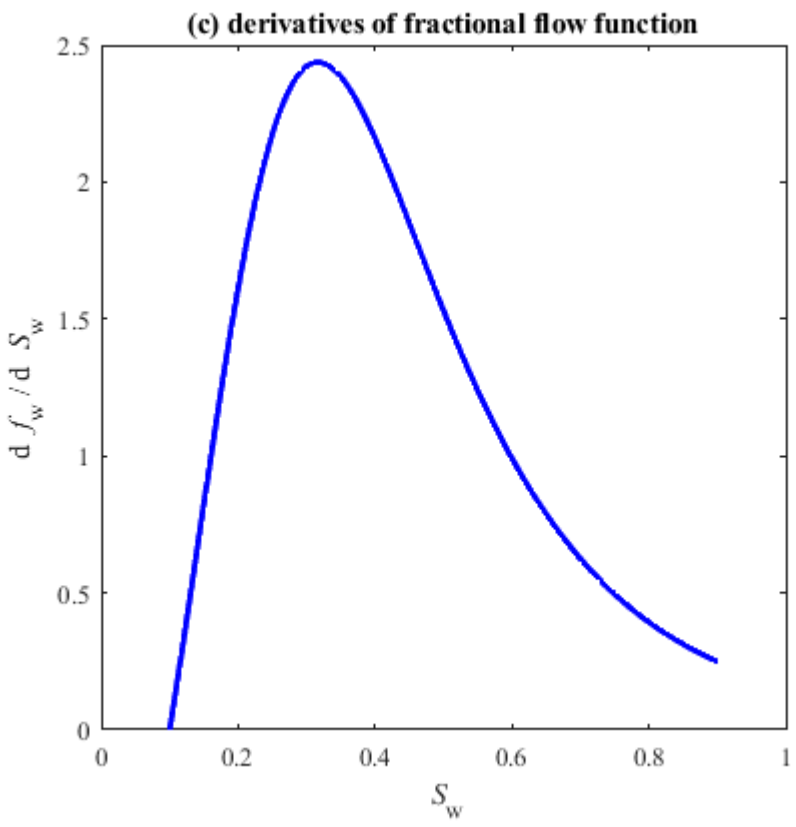
5.2 Plot the fractional flow function curve

```
h_fig2 = figure(2);
set(h_fig2, 'color', 'w', 'NumberTitle', 'off', 'Name', 'Fractional Flow Curve fw');
plot(Sw, fw, '-b', 'LineWidth', 2.0);
hold on;
plot(Sw, (dfwf .* Sw + bf), '-r', 'LineWidth', 2.0);
plot(Swc, 0, 'ro', 'Markersize', 8);
plot(Swf, fw(index), 'ro', 'Markersize', 8);
plot([Swf Swf], [0 fw(index)], '--r', 'LineWidth', 1.5);
plot([0 Swf], [fw(index) fw(index)], '--r', 'LineWidth', 1.5);
axis([0.0 1.0 0.0 1.0]);
axis square;
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('fractional flow function curve')
xlabel('{\it S}_w');
ylabel('{\it f}_w');
set(gca, 'YTick', 0:0.2:1);
```



5.3 Plot the derivate curve of fractional flow function

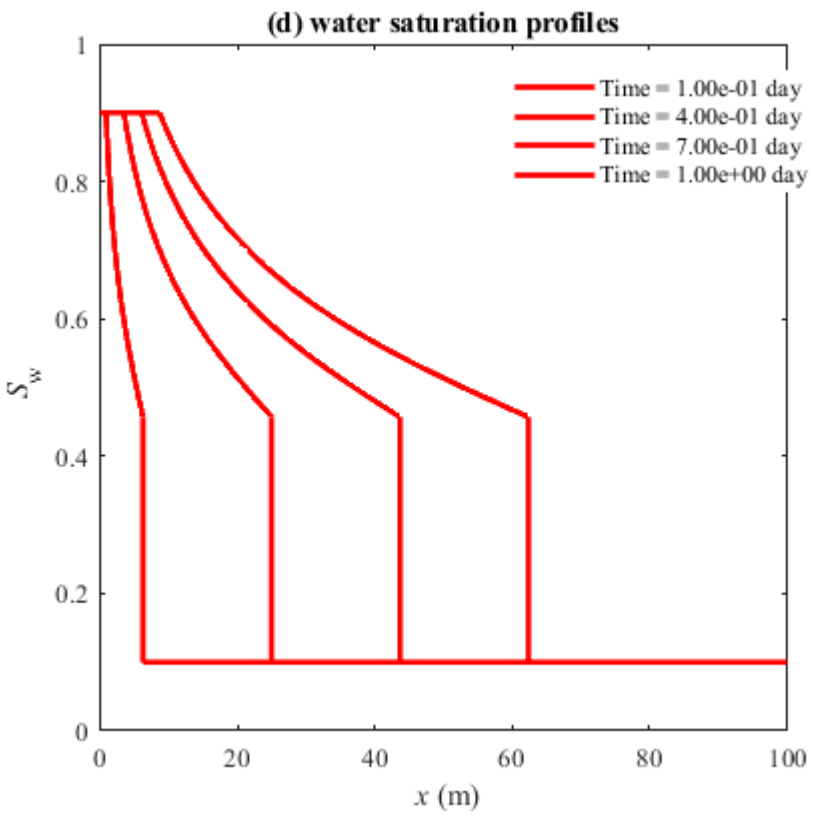
```
h_fig3 = figure(3);
set(h_fig3, 'color', 'w', 'NumberTitle', 'off', 'Name', 'Derivatives of Fractional Flow Curve dfw/dSw');
plot(Sw, dfw, '-b', 'LineWidth', 2.0);
set(gca, 'XLim', [0 1]);
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('(c) derivatives of fractional flow function')
xlabel('{\it S}_w');
ylabel('d {\it f}_w / d {\it S}_w');
axis square;
```



5.4 Plot the water saturation profiles

```
h_fig4 = figure(4);
set(h_fig4, 'color', 'w', 'NumberTitle', 'off', 'Name', 'Water Saturation Profiles');

SwTime = [];

for ti = 1:nts
    plot(Xsw(:, ti), Swt, '-r', 'LineWidth', 2.0);
    hold on;
    plot([0 Xsw(1, ti)], [1 - Sor 1 - Sor], '-r', 'LineWidth', 2.0);
    SwTime = [SwTime; ['Time = ' num2str(t(ti) / 86400, format) ' day']];
    plot([Xsw(end, ti) Xsw(end, ti)], [Swf Swc], '-r', 'LineWidth', 2.0);
    plot([Xsw(end, ti) L], [Swc Swc], '-r', 'LineWidth', 2.0);
end

set(gca, 'YLim', [0 1], 'YTick', 0:0.2:1, 'XLim', [0 L]);
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('(d) water saturation profiles')
xlabel('{\it x} (m)');
ylabel('{\it S}_w');
h_legend2 = legend(SwTime);
set(h_legend2, 'Box', 'on', 'Location', 'best');
```

```
axis square;
```




```
%end
```

A.2 MATLAB CODE OF INTEGRAL METHOD BASED ON MASS BALANCE PRINCIPLE

Problem description

Analytical solution of Buckley–Leverett equation using integral method wetting phase fluid displacing non-wetting phase fluid Brooks–Corey relative permeability curve has been applied

```
close all; clear all; clc;
```

1 Parameters initialization

1.1 rock properties

```
L = 5.0; % domain length [m]
A = 1.0; % area of cross-section [m^2]
phi = 0.30; % porosity [-]
k = 9.869e-12; % absolute permeability [m^2]
theta = pi * 0.0; % angle: x-direction vs horizontal [rad]
```

1.2 fluid properties

```
muo = 5.0e-3; % oil phase viscosity [Pa*s]
muw = 1.0e-3; % water phase viscosity [Pa*s]
rho_o = 0.8e3; % oil density [kg/m^3]
rho_w = 1.0e3; % water density [kg/m^3]
dlt_rho = rho_w - rho_o; % density difference [kg/m^3]
% 1.3 relative permeability parameters
Sor = 0.20; % residual oil saturation
Swc = 0.20; % connate water saturation
no = 2.00; % exponent of oil phase
nw = 2.00; % exponent of water phase
kro_max = 0.75; % maximum permeability of oil
krw_max = 0.75; % maximum permeability of water
```

1.4 other initial parameters

```
dlt_Sw = 1e-3; % constant saturation step
Sw = [(Swc + eps):dlt_Sw:(1 - Sor)]'; % water saturation vector
qt = 5.0e-4; % constant water injection rate [m^3/s]
g = 9.8067; % gravity acceleration constant [m/s^2]
time0 = 3600 * 0.02; % initial calculation time [s]
timef = 3600 * 0.20; % final calculation time [s]
nts = 4; % time steps for calculating
nsw = size(Sw, 1); % number of wetting saturation vector
format = '%4.2e'; % precision format for plotting legend
```

2 Relative permeabilities, fractional flow function and its derivative

2.1 initialization of function handles

```
kr_w = @(S) krw_max .* ((S - Swc) / (1 - Swc - Sor)) .^ nw; % relative permeability of water
kr_o = @(S) kro_max .* ((1 - S - Sor) / (1 - Swc - Sor)) .^ no; % relative permeability of oil
mob = @(S) (kr_o(S) ./ muo) .* (muw ./ kr_w(S)); % mobility function
f_w = @(S) 1 ./ (1 + mob(S)) - A * k .* kr_o(S) ./ (qt * muo) * ... % fractional flow function fw
dlt_rho * g * sin(theta) ./ (1 + mob(S));
df_w = @(S) ((nw .* mob(S) ./ (S - Swc)) + (no .* mob(S)) ./ (1 - S - Sor))... % derivative of fw
./ (1 + mob(S)) .^ 2 + A * k .* kr_o(S) * no * dlt_rho * g * sin(theta) ./ ...
((1 - S - Sor) * qt * muo .* (1 + mob(S))) + A * k .* kr_o(S) * dlt_rho * g * sin(theta) ./ ...
```

```
(qt * muo .* (1 + mob(S)) .^ 2) .* ...
(((nw .* mob(S) ./ (S - Swc)) + (no .* mob(S)) ./ (1 - S - Sor)) ./ (1 + mob(S)) .^ 2);
```

2.2 relative permeability and fractional flow vectors

```
krw = kr_w(Sw); % relative permeability vector of water
kro = kr_o(Sw); % relative permeability vector of oil
fw = f_w(Sw); % fractional flow vector
dfw = df_w(Sw); % fractional flow derivate vector
```

3 Calculate advance frontal saturation and travelling distance

3.1 calculate travelling distance of every saturations

```
t = linspace(time0, timef, nts);
dfwt = dfw(end:-1:1); % inverted sequence of dfw
fwt = fw(end:-1:1); % inverted sequence of fw
Swt = Sw(end:-1:1); % inverted sequence of Sw

for ti = 1:nts
    Wi(ti) = qt * t(ti); % injected wetting phase fluid volume

    for i = 1:nsw
        Xsw(i, ti) = qt * t(ti) / (A * phi) * dfwt(i); % calculate travelling distance
    end

    Xsw0 = [0; Xsw(1:end-1,ti)]; % the 2nd travelling distance vector
    dlt_Xsw = Xsw(:, ti) - Xsw0; % dlt_Xsw = Xsw_j - Xsw_(j-1), x_0 = 0
    dlt_Swt = Swt - Swc; % dlt_Swt = Swt,j - Swc

    for i = 1:nsw
        % calculate the injected fluid volume from 0 to Xswf
        Vi = A * phi * sum(dlt_Xsw(1:i) .* dlt_Swt(1:i));

        if Vi >= Wi(ti)
            index(ti) = i; % index of the Swf in Swt
            Swf(ti) = Swt(i); % advance front saturation: Swf
            Xswf(ti) = Xsw(i, ti); % the position of Swf
            break;
        end

    end

end

Xsw = Xsw(1:index(1), :);
Swt = Swt(1:index(1));
```

3.2 Calculate the time when Swf reaches at production well

```
krw_swf = kr_w(Swf(1));
kro_swf = kr_o(Swf(1));
fw_swf = f_w(Swf(1));
dfw_swf = df_w(Swf(1));
t_pw = A * phi * L / (qt * dfw_swf);
```

3.3 calculate pressure profiles

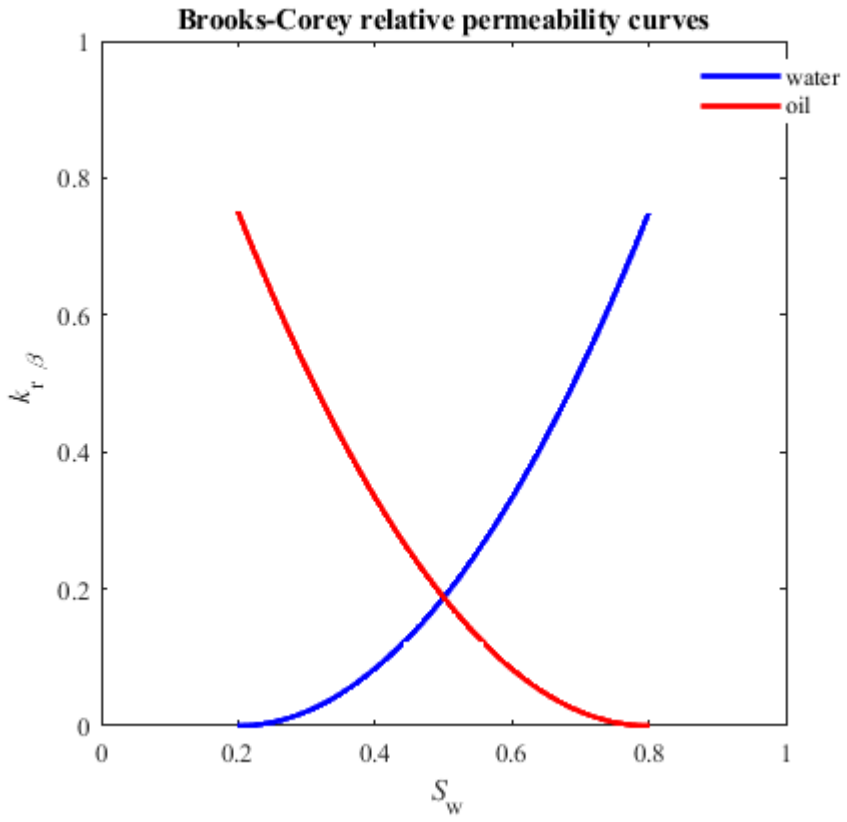
```
P_pw = 1e5;
dlt_x = L / 1e2;
x = [L:-dlt_x:0]';
P = zeros(size(x, 2), nts);
P(1, :) = P_pw;

for ti = 1:nts

    if Xswf(ti) <= L

        for i = 2:size(x, 1)
            Swx = calObjFun(x(i), Swt, Xsw(:, ti));

            if Swx <= Swf(ti)
                fwx = 0;
                krox = kr_o(Swc);
                P(i, ti) = dlt_x * (qt * (1 - fwx) * muo / (A * k * krox) - rho_o * g * sin(theta)) + P(i - 1, ti);
            else
                fwx = calObjFun(Swx, fwt, Swt);
                krox = kr_o(Swx);
                P(i, ti) = dlt_x * (qt * (1 - fwx) * muo / (A * k * krox) - rho_o * g * sin(theta)) + P(i - 1, ti);
            end

        end

    else
```

```
        for i = 2:size(x, 1)
            Swx = calObjFun(x(i), Swt, Xsw(:, ti));
            fwx = calObjFun(Swx, fwt, Swt);
            krox = kr_o(Swx);
            P(i, ti) = dlt_x * (qt * (1 - fwx) * muo / (A * k * krox) - rho_o * g * sin(theta)) + P(i - 1, ti);
        end

    end

end
```

4 Plot results

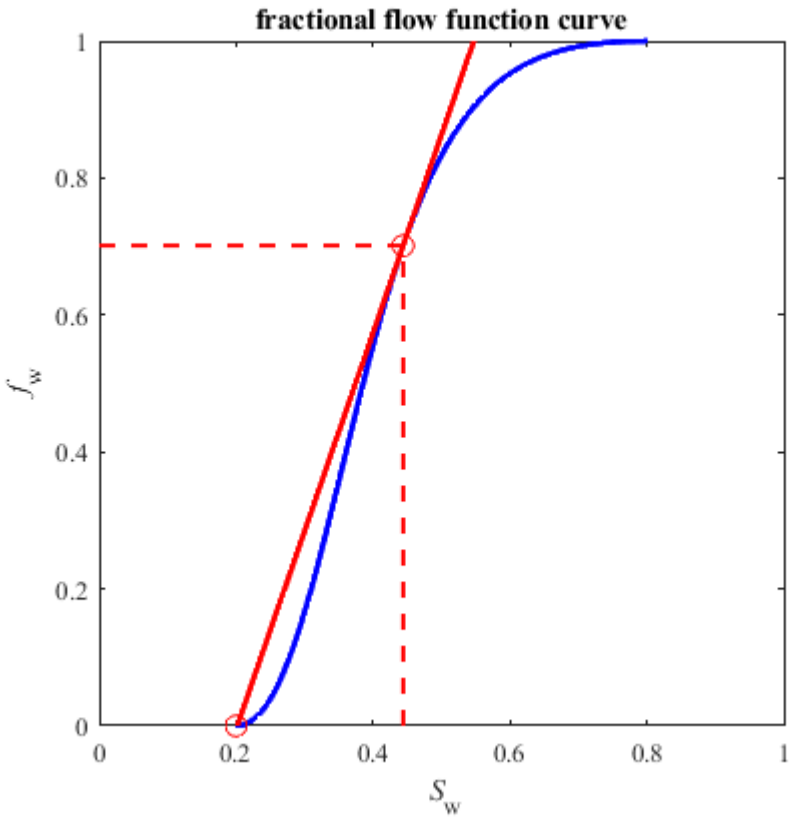
4.1 Plot the relative permeability curves

```
h_fig1 = figure(1);
set(h_fig1, 'color', 'w', 'NumberTitle', 'off', 'Name', 'Relative Permeability Curves');
plot(Sw, krw, '-b', Sw, kro, '-r', 'LineWidth', 2.0);
axis([0.0 1.0 0.0 1.0]);
axis square;
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('Brooks-Corey relative permeability curves')
xlabel('{\it S}_w');
ylabel('{\it k}_{r {\it \beta}}');
set(gca, 'YTick', 0:0.2:1);
h_legend1 = legend('water', 'oil');
set(h_legend1, 'Box', 'on', 'Location', 'best');
```



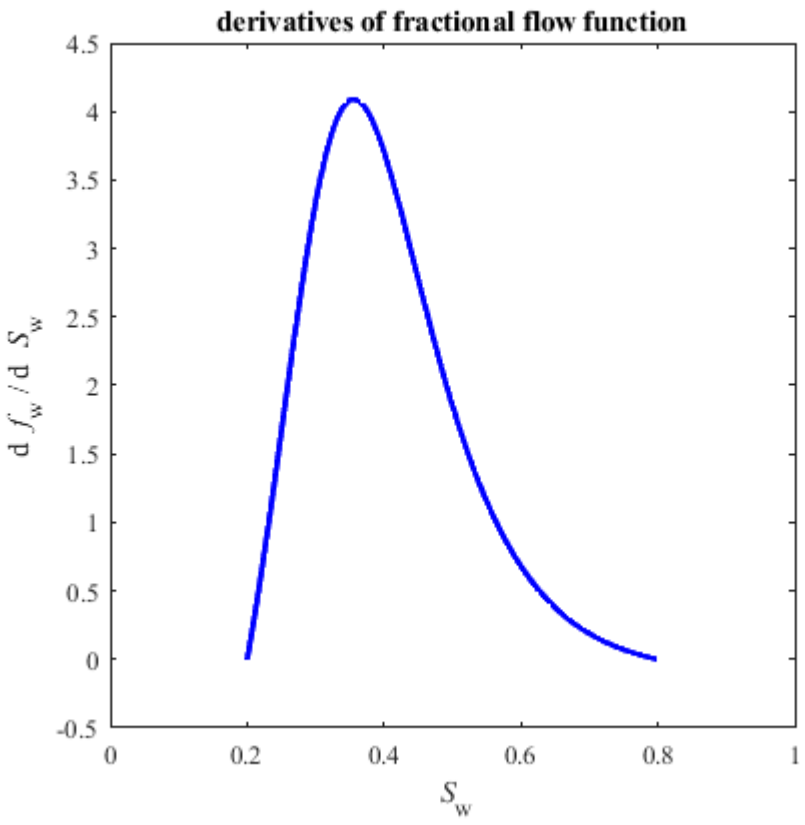
4.2 Plot the fractional flow function curve

```
h_fig2 = figure(2);
set(h_fig2, 'color', 'w', 'NumberTitle', 'off', 'Name', 'Fractional Flow Curve fw');
plot(Sw, fw, '-b', 'LineWidth', 2.0);
hold on;
b = fw_swf - dfw_swf * Swf(1);
plot(Sw, (dfw_swf .* Sw + b), '-r', 'LineWidth', 2.0);
plot(Swc, 0, 'ro', 'Markersize', 8);
plot(Swf(1), fw_swf, 'ro', 'Markersize', 8);
plot([Swf(1) Swf(1)], [0 fwt(index(1))], '--r', 'LineWidth', 1.5);
plot([0 Swf(1)], [fwt(index(1)) fwt(index(1))], '--r', 'LineWidth', 1.5);
axis([0.0 1.0 0.0 1.0]);
axis square;
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('fractional flow function curve')
xlabel('{\it S}_w');
ylabel('{\it f}_w');
set(gca, 'YTick', 0:0.2:1);
```



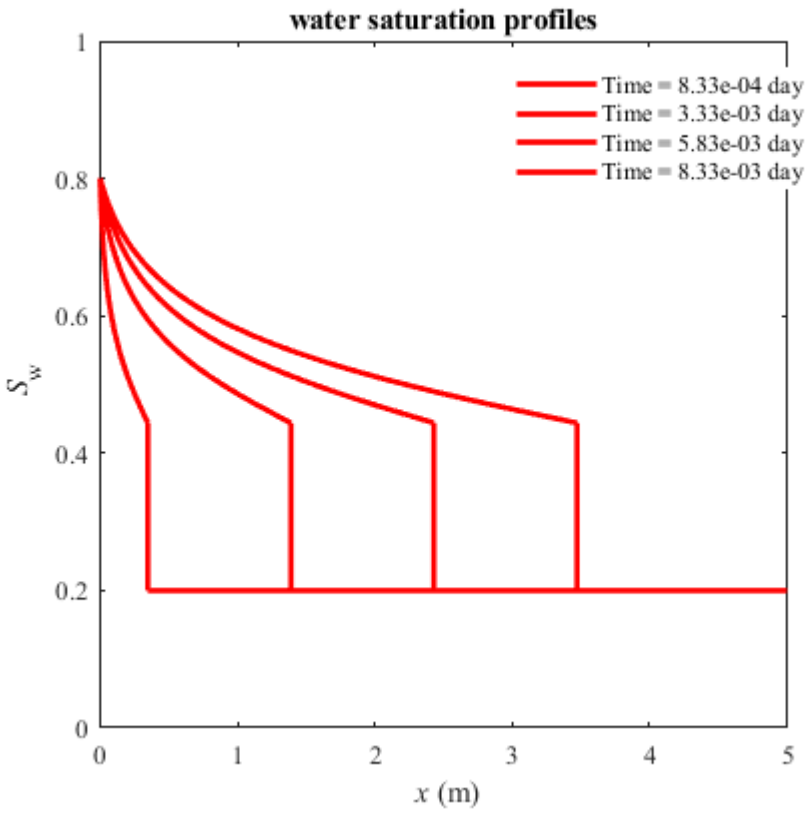
4.3 Plot the derivative curve of fractional flow function

```
h_fig3 = figure(3);
set(h_fig3, 'color', 'w', 'NumberTitle', 'off', 'Name', 'Derivatives of Fractional Flow Curve dfw / dSw');
plot(Sw, dfw, '-b', 'LineWidth', 2.0);
set(gca, 'XLim', [0 1]);
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('derivatives of fractional flow function')
xlabel('{\it S}_w');
ylabel('d {\it f}_w / d {\it S}_w');
axis square;
```



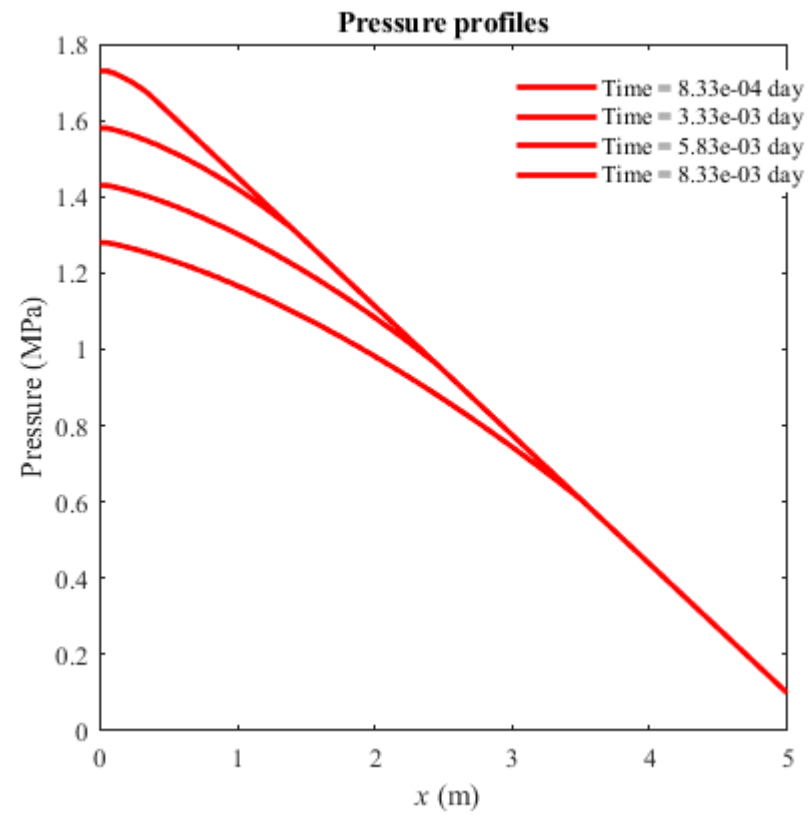
4.4 Plot the water saturation profiles

```
h_fig4 = figure(4);
set(h_fig4, 'color', 'w', 'NumberTitle', 'off', 'Name', 'water Saturation Profiles:Sw(t)');
SwTime = [];

for ti = 1:nts
    plot(Xsw(:, ti), Swt, '-r', 'LineWidth', 2.0);
    hold on;
    plot([0 Xsw(1, ti)], [1 - Sor 1 - Sor], '-r', 'LineWidth', 2.0);
    SwTime = [SwTime; ['Time = ' num2str(t(ti) / 86400, format) ' day']];
    plot([Xsw(end, ti) Xsw(end, ti)], [Swf(ti) Swc], '-r', 'LineWidth', 2.0);
    plot([Xsw(end, ti) L], [Swc Swc], '-r', 'LineWidth', 2.0);
end

set(gca, 'YLim', [0 1], 'YTick', 0:0.2:1, 'XLim', [0 L]);
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('water saturation profiles')
xlabel('{\it x} (m)');
ylabel('{\it S}_w');
h_legend4 = legend(SwTime);
set(h_legend4, 'Box', 'on', 'Location', 'best');
axis square;
```



4.5 Plot the pressure profiles

```
h_fig5 = figure(5);
set(h_fig5, 'color', 'w', 'NumberTitle', 'off', 'Name', 'Pressure Profiles: P(t)');
SwTime = [];

for ti = 1:nts
    plot(x, P(:, ti) / 1e6, '-r', 'LineWidth', 2.0);
    hold on;
    SwTime = [SwTime; ['Time = ' num2str(t(ti) / 86400, format) ' day']];
end

set(gca, 'XLim', [0 L]);
set(gca, 'Fontname', 'Times New Roman', 'FontSize', 10);
title('Pressure profiles')
xlabel('{\it x} (m)');
ylabel('Pressure (MPa)');
h_legend5 = legend(SwTime);
set(h_legend5, 'Box', 'on', 'Location', 'best');
axis square;
```




```
%end
```

AUXILIARY FUNCTIONS

```
function [index, Swf, dfwf, bf] = calSatFront(Sw, fw, dfw)
    %% calculate advance front saturation using Welge graphic method
    % Sw --- saturation vector of displacing wetting phase fluid
    % fw --- fractional flow vector of displacing wetting phase fluid
    % dfw --- fractional flow derivate vector of displacing wetting phase fluid
    % index --- index of Swf in Sw
    % Swf --- advance front saturation
    % dfwf --- fractional flow derivative at Swf
    % bf --- intercept value for tangent line
```

```
index = 0;
ns = size(Sw, 1);
df = 1 / (Sw(ns) - Sw(1));
dlt_Sw = Sw(ns) - Sw(ns - 1);

if nargin == 2
    % calculate derivate
    for i = 3:(ns - 2)
        dfw(i) = (fw(i + 1) - fw(i - 1)) / (2 * dlt_Sw);
    end

    dfw(2) = (-11 * fw(2) + 18 * fw(3) - 9 * fw(4) + 2 * fw(5)) / (6 * dlt_Sw);
    dfw(1) = abs(2 * dfw(2) - dfw(3));
    dfw(ns - 1) =- (-11 * fw(nt - 1) + 18 * fw(nt - 2) - 9 * fw(nt - 3) + 2 * fw(nt - 4)) / (6 * dlt_Sw);
    dfw(ns) = abs(2 * dfw(nt - 1) - dfw(nt - 2));
    % calculate Swf and its index
    for i = 2:ns - 1
        dfds = fw(i) / (Sw(i) - Sw(1));

        if (dfds < dfw(i - 1)) && (dfds > dfw(i + 1)) && (dfds >= df)
            index = i;
            Swf = Sw(i);
            dfwf = dfds;
            bf = fw(i) - dfwf * Sw(i);
            break;
        end
    end

    if index == 0
        error('Cant find a tangent line through point (Swc,0). Decrease dlt_Sw!');
    end

elseif nargin == 3
    % calculate Swf and its index
    for i = 2:ns - 1
        dfds = fw(i) / (Sw(i) - Sw(1));

        if (dfds < dfw(i - 1)) && (dfds > dfw(i + 1)) && (dfds >= df)
            index = i;
            Swf = Sw(i);
            dfwf = dfds;
            bf = fw(i) - dfwf * Sw(i);
            break;
        end

    end

    if index == 0
        error('Cant find a tangent line through point (Swc,0). Decrease dlt_Sw!');
    end

else
    error('the number of input arguments in function calSatFront is incorrect!');
end
%end function calSatFront()
```

```
function obj_fun = calObjFun(obj_var, Fun, Var, index_var)
    %% calculate obj_fun at obj_var using interpolation method
    % Fun --- basic function value vector
    % Var --- basic variable value vector
    % obj_var --- objective variable value vector
    % obj_fun --- objective function value vector
    % index --- optional for non-monotone Fun
    ns = size(obj_var, 1);
    obj_fun = zeros(ns, 1);

    if nargin == 3

        for i = 1:ns
            [w, index] = sort(abs(Var - obj_var(i)));
            obj_fun(i) = Fun(index(1));
        end

    elseif nargin == 4

        for i = 1:ns
            [w, index] = sort(abs(Var - obj_var(i)));

            if index(1) >= index_var
```

```
                obj_fun(i) = Fun(index(1));
            else
                obj_fun(i) = Fun(index(2));
            end

        end

    else
        error('Wrong input parameters in calObjFun function!');
    end

end
%end function calObjFun()
```